

Neural Multiclass Document/Text Classifier

RNN Sequence Classification, BERT Representations, and PyTorch

Yasser Hifny

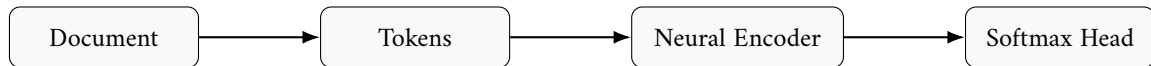
Faculty of Computers and Artificial Intelligence
Capital University

1. From a Document to a Class

Goal

Given a document x and a set of K classes, predict one class:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} P(y = k \mid x).$$



The encoder converts a variable-length sequence into a representation that can be used for classification.

2. Multiclass Neural Classification

For K mutually exclusive classes, the classifier produces K logits:

$$\mathbf{z} = W\mathbf{r} + \mathbf{b}, \quad \mathbf{z} \in \mathbb{R}^K.$$

The softmax converts logits into class probabilities:

$$P(y = k \mid x) = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}.$$

Cross-Entropy Loss

For a training example with true class y :

$$\mathcal{L} = -\log P(y \mid x).$$

Key distinction

The classification head is usually simple. Most of the modeling power comes from the neural encoder that produces $\mathbf{r}$.

3. Why Use an RNN for Text?

A document is naturally represented as an ordered sequence:

$$x_1, x_2, \dots, x_T.$$

An RNN processes tokens sequentially while maintaining a hidden state:

$$\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_T.$$

Sequence Classification

For document classification, the final state $\mathbf{h}_T$ can summarize the sequence and feed a classification head:

$$x_1, \dots, x_T \rightarrow \mathbf{h}_T \rightarrow \mathbf{z} \rightarrow \text{softmax}(\mathbf{z})$$

This is the basic RNN document-classification architecture.

4. Vanilla RNN Equations

Let $\mathbf{x}_t \in \mathbb{R}^d$ be the embedding of token t and $\mathbf{h}_t \in \mathbb{R}^m$ the hidden state.
The vanilla RNN update is:

$$\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

where

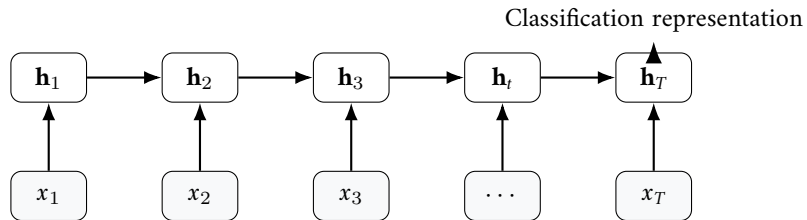
$$W_{xh} \in \mathbb{R}^{m \times d}, \quad W_{hh} \in \mathbb{R}^{m \times m}.$$

The initial state is commonly

$$\mathbf{h}_0 = \mathbf{0}.$$

The recurrence gives the model a memory of preceding tokens.

5. RNN Unrolled Through Time



The same RNN parameters are reused at every time step:

$$\mathbf{h}_t = f_{\theta}(\mathbf{x}_t, \mathbf{h}_{t-1}).$$

6. RNN Sequence Classifier: Last State

After processing all T tokens, use the last hidden state:

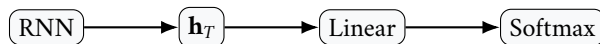
$$\mathbf{r} = \mathbf{h}_T.$$

The classification head is:

$$\mathbf{z} = W_c \mathbf{h}_T + \mathbf{b}_c,$$

followed by

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}).$$



Important

The last state is a fixed-size vector, regardless of document length.

7. RNN Classification: PyTorch Structure

A typical PyTorch model contains:

- 1 an embedding layer,
- 2 an RNN,
- 3 extraction of the last hidden state,
- 4 a linear classification head.

PyTorch model structure

```
self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_id)
self.rnn = nn.RNN(emb_dim, hidden_dim, batch_first=True)
self.classifier = nn.Linear(hidden_dim, num_classes)

def forward(self, x):
    e = self.embedding(x)
    _, h = self.rnn(e)
    h_last = h[-1]
    return self.classifier(h_last)
```

The output has shape:

$[\text{batch size}, K]$.

8. RNN Variants for Document Classification

The simple RNN can be replaced by gated recurrent architectures.

Model	Main mechanism	Classification vector
Vanilla RNN	recurrent tanh state	$\mathbf{h}_T$
LSTM	input/forget/output gates	final hidden state
GRU	update/reset gates	final hidden state
BiRNN	forward + backward context	concatenated final states

For a bidirectional RNN:

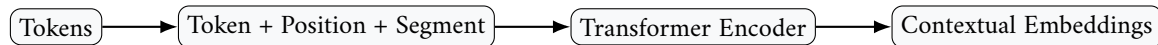
$$\mathbf{r} = [\vec{\mathbf{h}}_T; \overleftarrow{\mathbf{h}}_1].$$

Limitation

A single recurrent state must compress a long document into a fixed-size vector.

9. From RNNs to Transformer Encoders

BERT replaces recurrent processing with Transformer encoder layers.



Self-attention allows each token representation to interact with other tokens.

$$\mathbf{H} = [\mathbf{h}_1, \dots, \mathbf{h}_T], \quad \mathbf{h}_t \in \mathbb{R}^d.$$

10. BERT at a High Level

BERT stands for **Bidirectional Encoder Representations from Transformers**.

Architecture

BERT is a stack of Transformer encoder layers trained to produce contextual representations of tokens.

For an input sequence

$$[\text{CLS}], x_1, \dots, x_T, [\text{SEP}],$$

BERT produces:

$$H = [\mathbf{h}_{\text{CLS}}, \mathbf{h}_1, \dots, \mathbf{h}_T, \mathbf{h}_{\text{SEP}}].$$

Each $\mathbf{h}_t$ depends on the surrounding context.

11. BERT Input Representation

The original BERT input representation combines three components:

$$\mathbf{e}_t = \mathbf{e}_t^{token} + \mathbf{e}_t^{position} + \mathbf{e}_t^{segment}$$

- **Token embeddings:** represent WordPiece tokens.
- **Position embeddings:** indicate token position.
- **Segment embeddings:** distinguish sentence A from sentence B.

The resulting sequence is passed through the Transformer encoder stack.

12. The [CLS] Token

BERT adds a special token at the beginning:

$$[\text{CLS}], x_1, x_2, \dots, x_T, [\text{SEP}].$$

After the Transformer encoder:

$$\mathbf{h}_{CLS} = \text{BERT}([\text{CLS}], x_1, \dots, x_T, [\text{SEP}])_{CLS}.$$

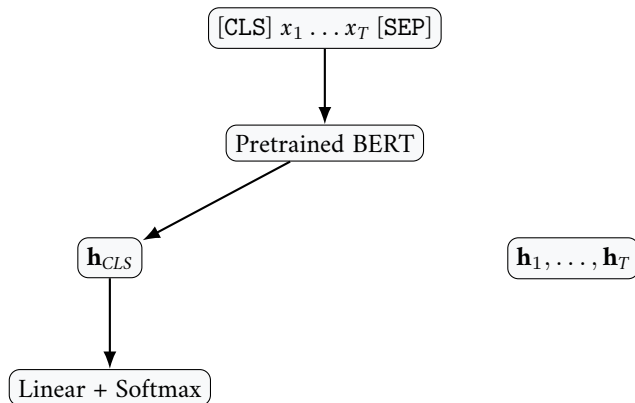
Classification Representation

For sequence-level classification, $\mathbf{h}_{CLS}$ is commonly used as a fixed-size representation of the complete input.

The classification head is:

$$\mathbf{z} = W_c \mathbf{h}_{CLS} + \mathbf{b}_c.$$

13. BERT + [CLS] Classification



Only the selected representation is connected to the classification head in this setup.

$$P(y = k \mid x) = \text{softmax}(W_c \mathbf{h}_{\text{CLS}} + \mathbf{b}_c)_k.$$

14. Mean Pooling of BERT Token Embeddings

Instead of using only the [CLS] vector, average token representations.

For valid token positions $1, \dots, T$:

$$\mathbf{r}_{mean} = \frac{1}{T} \sum_{t=1}^T \mathbf{h}_t$$

Then:

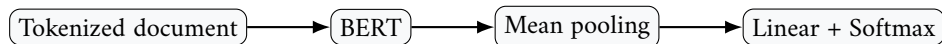
$$\mathbf{z} = W_c \mathbf{r}_{mean} + \mathbf{b}_c.$$

Padding

For padded batches, the mean must ignore padding tokens. If $m_t \in \{0, 1\}$ is the attention mask:

$$\mathbf{r}_{mean} = \frac{\sum_t m_t \mathbf{h}_t}{\sum_t m_t}.$$

15. BERT + Mean-Pooled Classification



If

$$H = [\mathbf{h}_{CLS}, \mathbf{h}_1, \dots, \mathbf{h}_T, \mathbf{h}_{SEP}],$$

mean pooling is applied to the selected non-padding token representations.

Interpretation

Mean pooling aggregates information distributed across all contextualized token representations.

16. [CLS] vs. Mean Pooling

	[CLS] representation	Mean pooling
Representation	$\mathbf{h}_{CLS}$	Average of selected token embeddings
Computation	Very simple	Requires masked aggregation
Information	Uses the learned [CLS] representation	Explicitly aggregates all token states
Classifier input	Fixed dimension d	Fixed dimension d
Assignment	Train one classifier using [CLS]	Train another classifier using mean pooling

Experimental question

Which representation works better on the SANAD document-classification task?

17. BERT Self-Attention: Why Context Matters

For one attention head:

$$Q = HW^Q, \quad K = HW^K, \quad V = HW^V.$$

Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

A token can therefore attend to other tokens in the sequence.

For document classification

The contextualized representation of a word can incorporate information from distant words, rather than relying on a recurrent chain of states.

18. BERT Pre-training: Masked Language Modeling

BERT is pretrained before being fine-tuned for downstream tasks.

Masked Language Modeling (MLM)

Random input tokens are selected and the model learns to predict the original tokens:

$$x_1, \dots, \boxed{x_t}, \dots, x_T \rightarrow x_1, \dots, [\text{MASK}], \dots, x_T.$$

The MLM objective can be written as:

$$\mathcal{L}_{MLM} = - \sum_{t \in M} \log P(x_t \mid x_{\setminus M}),$$

where M is the set of selected masked positions.

This teaches BERT contextual representations.

19. BERT Pre-training: Next Sentence Prediction

The original BERT pre-training procedure also used **Next Sentence Prediction (NSP)**. Given two segments A and B , predict whether B follows A in the source corpus.

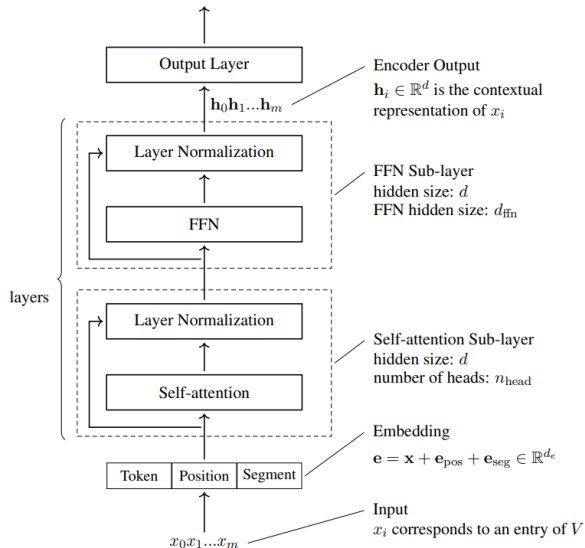


Combined original BERT pre-training

$$\mathcal{L} = \mathcal{L}_{MLM} + \mathcal{L}_{NSP}.$$

Later BERT-family models do not necessarily use NSP, so the pre-training objective depends on the particular pretrained model.

BERT model



BERT model: MLM

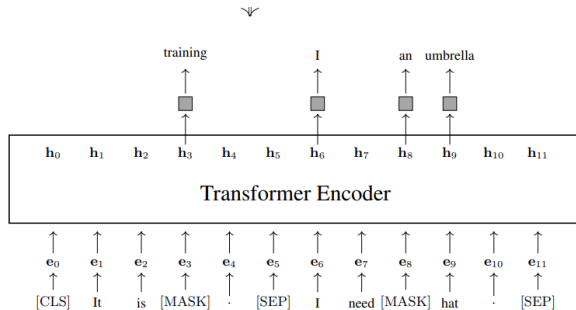


Figure: BERT model MLM from <https://arxiv.org/pdf/2501.09223>.

BERT model: Pre-training

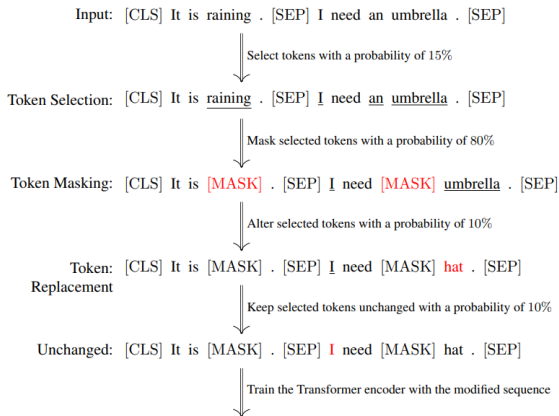


Figure: BERT model pre-training details from <https://arxiv.org/pdf/2501.09223>.

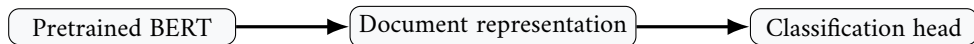
20. From Pre-training to Fine-tuning

Stage 1: Pre-training

BERT is initialized from large-scale unlabeled text and learns contextual language representations.

Stage 2: Fine-tuning

Attach a task-specific linear classification head to a fixed-size document representation.



The assignment compares two representations: the [CLS] embedding and the masked mean of token embeddings.

21. Fine-tuning Strategies

There are several practical choices.

- 1 **Feature extraction:** freeze BERT and train only the classification head.
- 2 **Partial fine-tuning:** unfreeze selected BERT layers.
- 3 **Full fine-tuning:** update BERT and the classification head jointly.

End-to-end objective

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N -\log P_{\theta}(y_i | x_i).$$

For the assignment, compare [CLS] and mean-pooling representations under the same evaluation protocol.

22. PyTorch + Pretrained BERT: Model Skeleton

A Hugging Face BERT encoder can be used inside a PyTorch classifier.

PyTorch BERT classifier

```
from transformers import AutoModel
import torch.nn as nn

class BertClassifier(nn.Module):
    def init(self, model_name, num_classes):
        self.bert = AutoModel.from_pretrained(model_name)
        d = self.bert.config.hidden_size
        self.classifier = nn.Linear(d, num_classes)

    def forward(self, input_ids, attention_mask):
        out = self.bert(input_ids=input_ids, attention_mask=attention_mask)
        cls = out.last_hidden_state[:, 0, :]
        logits = self.classifier(cls)
        return logits
```

The index 0 corresponds to the [CLS] position for BERT-style inputs.

23. Mean-Pooling Implementation

Use the attention mask to exclude padding tokens.

Masked mean pooling in PyTorch

```
hidden = out.last_hidden_state
mask = attention_mask.unsqueeze(-1).float()
summed = (hidden * mask).sum(dim=1)
count = mask.sum(dim=1).clamp(min=1e-9)
mean_embedding = summed / count
logits = self.classifier(mean_embedding)
```

Mathematically:

$$\mathbf{r}_{mean}^{(b)} = \frac{\sum_{t=1}^T m_t^{(b)} \mathbf{h}_t^{(b)}}{\sum_{t=1}^T m_t^{(b)}}.$$

Common implementation error

Do not include padded positions in the denominator or in the summed embeddings.

24. Tokenization and Batching

A BERT tokenizer converts raw text into:

`input_ids,` `attention_mask.`

Typical batch construction:

variable-length documents $\rightarrow$ truncate $\rightarrow$ pad $\rightarrow$ attention masks.

Important

BERT has a maximum input length. Long SANAD documents therefore require a consistent truncation strategy for the assignment.

A batch has shapes:

$\text{input_ids} \in \mathbb{N}^{B \times T},$ $\text{attention_mask} \in \{0, 1\}^{B \times T}.$

25. Training Loop for Multiclass Classification

For each batch:

- 1 Tokenize documents.
- 2 Move tensors to the device.
- 3 Run BERT and the classification head.
- 4 Compute cross-entropy loss.
- 5 Backpropagate.
- 6 Update parameters.

PyTorch training step

```
optimizer.zero_grad()
logits = model(input_ids, attention_mask)
loss = criterion(logits, labels)
loss.backward()
optimizer.step()
```

For multiclass classification:

```
criterion = nn.CrossEntropyLoss().
```

Do not apply softmax before 'CrossEntropyLoss'.

26. Evaluating a Neural Document Classifier

Use the validation set for model selection and the test set only for final evaluation.

Useful metrics include:

$$\text{Accuracy} = \frac{\text{\#correct predictions}}{\text{\#examples}}.$$

For multiclass problems, also report:

$$F1_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K F1_k.$$

Recommended evaluation

- Accuracy
- Macro-F1
- Per-class precision, recall, and F1
- Confusion matrix

Compare RNN, BERT-[CLS], and BERT-mean only if the assignment asks for all three; otherwise focus on the two BERT representations.

27. Assignment: SANAD Neural Document Classifier

Objective

Implement a multiclass Arabic document classifier using the **SANAD dataset**, PyTorch, and a pretrained BERT model.

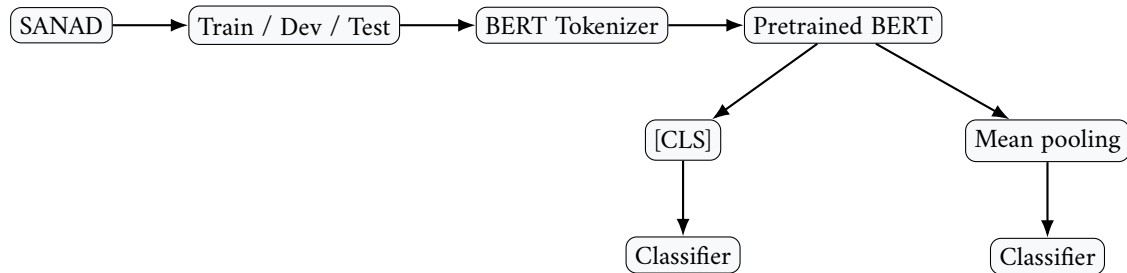
Required experiments

- 1 Build a PyTorch data pipeline for SANAD.
- 2 Tokenize documents with the selected pretrained BERT tokenizer.
- 3 Train a classifier using the **[CLS] embedding**.
- 4 Train a classifier using **masked mean pooling**.
- 5 Evaluate both approaches on the same test set.

Main research question

Does the [CLS] representation or the mean of contextual token embeddings provide better document-level classification performance?

28. Assignment: Recommended Pipeline



Keep preprocessing, tokenizer, maximum length, optimizer, and evaluation protocol consistent between the two BERT experiments.

29. Assignment: Deliverables

Submit:

- ❶ Complete PyTorch source code.
- ❷ Training and validation loss curves.
- ❸ Final test accuracy and Macro-F1.
- ❹ Confusion matrix.
- ❺ Per-class evaluation results.
- ❻ A comparison table for:
 - BERT + [CLS]
 - BERT + mean pooling
- ❼ A short discussion explaining which representation performed better and why.

Reproducibility

Record the pretrained model name, maximum sequence length, batch size, learning rate, number of epochs, random seed, and hardware used.

30. Key Takeaways

- An RNN processes text sequentially and can use its last hidden state for sequence classification.
- The RNN classifier maps $\mathbf{h}_T$ to class logits and applies softmax.
- BERT produces contextual token representations using Transformer encoders.
- The [CLS] representation is a natural fixed-size input to a classification head.
- Mean pooling aggregates contextual information across valid token positions.
- BERT is pretrained on large unlabeled text before downstream fine-tuning.
- The SANAD assignment compares [CLS] and mean-pooled BERT representations using PyTorch.

Text $\rightarrow$ BERT $\rightarrow$ Document Representation $\rightarrow$ Classifier $\rightarrow$ Class

Any Questions?

Neural Multiclass Document/Text Classifier